Inheritance - Reusing and Extending Functionality

Definition

Inheritance allows a class (child / subclass) to take properties and methods from another class (parent / superclass).

The new class, called a subclass (or derived class), can add or modify features of the parent class (or base class/superclass).

Benefits of Inheritance

Inheritance promotes code reuse and reduces redundancy.
It also makes it easier to extend programs without rewriting existing code.

Types of Inheritance:

There are following types of inheritance:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance

1. Single Inheritance:

Definition: Single inheritance occurs when a child class (subclass) inherits attributes and methods from exactly one parent class (superclass).

This is the most fundamental form of inheritance in Python, promoting code reusability by allowing the child class to extend or modify the behavior of the parent (as shown in Figure 3.3).

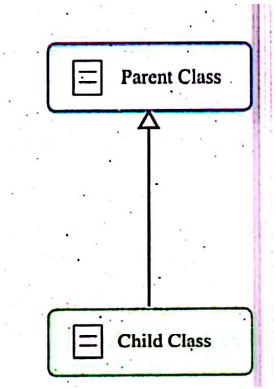


Figure 3.3: Single Inheritance

Example:
In this example, the Dog class inherits the basic functionality of an Animal but adds its own specific behavior.

```
# Parent class (Superclass)
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

# Child class (Subclass) inheriting from Animal
class Dog(Animal):
    def bark(self):
        print(f"{self.name} says Woof!")

# Create an object of the child class
my_dog = Dog("Buddy")

# Access method from Parent class
my_dog.eat()  # Output: Buddy is eating.

# Access method from Child class
my_dog.bark() # Output: Buddy says Woof!
```

TIDBIT

self is the reference to the current object, passed automatically to instance methods.

Important Terms

- **Parent / Base Class:** The class whose properties are inherited.
- **Child / Derived Class:** The class that inherits from the parent. It is defined by passing the parent class name in parenthesis: class Child(Parent):
- **Method Overriding:** A child class can provide a specific implementation of a method that is already provided by its parent.. You can use the super0 function to call the parent's version of that method.

Short Questions

1. What is inheritance?
2. What is a parent class and a child class?
3. Name the types of inheritance.
4. What is single inheritance?
5. What is method overriding?
6. In inheritance, how can we call the parent's version of that method that is override in child class?
7. **Why would a developer implement method overriding in a child class?**

    1. **Answer:** To provide a specific implementation of a method that already exists in the parent class.
8. **What function lets you reuse parent method logic inside an overridden child method?**
    1. **Answer:** The built-in `super()` function.

Long Questions

1. What is inheritance in OOP? What is a parent class and a child class? Explain its importance. Name the types of inheritance.
2. What is a Single Inheritance? Show with diagram. Write a Python program that explains single inheritance
3. **Explain the purpose of method overriding and the practical use of the `super()` function in Python inheritance.**
    1. **Answer:** Method overriding allows a child class to modify inherited parent behaviors to fit more specific use cases. For instance, a child class can override a generic `sound()` method from a base `Animal` class to output a specific "Bark" or "Meow" instead. If the child class needs to extend the parent's logic rather than completely replacing it, developers can call the `super()` function. This runs the parent's original method first, letting the child add custom steps on top without duplicating base code.